Pashov Audit Group

Ostium Security Review

September 7th 2025 - September 10th 2025

Contents

1. About Pashov Audit Group 3
2. Disclaimer 3
3. Risk Classification 3
4. About Ostium 4
5. Executive Summary 4
6. Findings 5

Medium findings 6

[M-01] Inflated trade notional in price impact calculations 6

Low findings 7

[L-01] `getTradeRolloverFeePure()` returns 1 not 0 when end is less than or equal to acc 7
[L-02] Incorrect `lastLongPure` validation 7
[L-03] Exposure limits check uses incorrect collateral amount 8
[L-04] `getPairRolloverFees()` returns outdated accumulated fees 8
[L-05] Underflow risk in `unregisterTrade()` when rollover fees equals collateral 8
[L-06] Incorrect assignment of `lastUpdateBlock` in `migrateRolloverFeesV2()` 9
[L-07] `pairMinLevPos` can be bypassed 10
[L-08] Trades fail as `registerTrade()` underflows from `builderFee` 11

1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over $100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work here or reach out on Twitter @pashovkrum.

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. About Ostium

Ostium is a decentralized perpetual trading protocol of Real World Assets (RWA). It works across commodities, Forex, cryptocurrencies, and a wide array of long-tail assets.

5. Executive Summary

A time-boxed security review of the **0xOstium/smart-contracts** repository was done by Pashov Audit Group, during which **0xunforgiven**, **IvanFitro**, **Tejas Warambhe**, **0xl33** engaged to review **Ostium**. A total of **9** issues were uncovered.

Protocol Summary	
Project Name	Ostium
Protocol Type	Perpetual DEX for RWA
Timeline	September 7th 2025 - September 10th 2025

Review commit hash:

- 70a6d108efe0313cb532dc0a436f1f736e3e9edf (0xOstium/smart-contracts)

Fixes review commit hash:

- 5c7030bc0ed3b0cc85e8bef964f112eaae92f85e (0xOstium/smart-contracts)

Scope

`OstiumPairInfos.sol` `OstiumPairsStorage.sol` `OstiumTrading.sol` `OstiumTradingCallbacks.sol` `OstiumTradingStorage.sol` `TradingCallbacksLib.sol` `TradingLib.sol` `interfaces/`

6. Findings

Findings count

Severity	Amount
Medium	1
Low	8
Total findings	**9**

Summary of findings

ID	Title	Severity	Status
[M-01]	Inflated trade notional in price impact calculations	Medium	Resolved
[L-01]	`getTradeRolloverFeePure()` returns 1 not 0 when end is less than or equal to acc	Low	Resolved
[L-02]	Incorrect `lastLongPure` validation	Low	Resolved
[L-03]	Exposure limits check uses incorrect collateral amount	Low	Acknowledged
[L-04]	`getPairRolloverFees()` returns outdated accumulated fees	Low	Acknowledged
[L-05]	Underflow risk in `unregisterTrade()` when rollover fees equals collateral	Low	Resolved
[L-06]	Incorrect assignment of `lastUpdateBlock` in `migrateRolloverFeesV2()`	Low	Resolved
[L-07]	`pairMinLevPos` can be bypassed	Low	Resolved
[L-08]	Trades fail as `registerTrade()` underflows from `builderFee`	Low	Resolved

Medium findings

[M-01] Inflated trade notional in price impact calculations

Severity

Impact: Medium

Likelihood: Medium

Description

The `getDynamicTradePriceImpact()` function uses inflated value of `tradeNotional` when called by `openTradeMarketCallback()` or `executeAutomationOpenOrderCallback()` because opening fees, oracle fee, and builder fee have not yet been subtracted from collateral. This inflated notional is used to calculate `priceImpactP` and update `decayedBuyVolume` / `decayedSellVolume`.

The issue causes multiple impacts: 1. Inflated price impact leads to less favorable execution prices, which also affects vault accounting through `accTotalPnl` updates. 2. Trades may be wrongly cancelled with `CancelReason.PRICE_IMPACT` in `getOpenTradeMarketCancelReason()` or `getAutomationOpenOrderCancelReason()` functions. 3. `DynamicSpreadState` is impacted by the incorrect values of `decayedBuyVolume` / `decayedSellVolume`, so future price impact calculations are also affected.

The calculated `tradeNotional` is not inflated when calling functions are `getHandleRemoveCollateralCancelReason()` or `getTradeAndPriceData()`, since fees are already paid at that point.

Recommendations

Modify `getDynamicTradePriceImpact()` to account for oracle, builder, and opening fees when calculating `tradeNotional`, but only when called by `openTradeMarketCallback()` or `executeAutomationOpenOrderCallback()` functions, since fees are not yet paid at that point.

Low findings

[L-01] `getTradeRolloverFeePure()` returns 1 not 0 when end is less than or equal to acc

`getTradeRolloverFeePure()` is used to calculate the rollover fee that a user needs to pay:

```
function getTradeRolloverFeePure(
    uint256 accRolloverFeesPerCollateral,
    uint256 endAccRolloverFeesPerCollateral,
    uint256 collateral,
    uint32 leverage
) public pure returns (uint256) {
    if (endAccRolloverFeesPerCollateral <= accRolloverFeesPerCollateral) {
        return 1;
    }
    uint256 rolloverFee = ((endAccRolloverFeesPerCollateral - accRolloverFeesPerCollateral) *
collateral * leverage)
        / PRECISION_18 / PRECISION_2;

    return (rolloverFee > collateral) ? collateral : rolloverFee;
}
```

When `endAccRolloverFeesPerCollateral <= accRolloverFeesPerCollateral`, the function returns `1` instead of `0`, even though the user has not accrued any rollover fees yet.

Recommendation: Return `0` when `endAccRolloverFeesPerCollateral <= accRolloverFeesPerCollateral` to accurately reflect that no rollover fees are owed.

[L-02] Incorrect `lastLongPure` validation

`updateRolloverFees()` is used to update the rollover fees, and the new rollover fees must not exceed `maxRolloverFeePerBlock`:

```
if (absRolloverFee > r.maxRolloverFeePerBlock) {
    revert WrongParams();
}
```

The issue is that in `setPairRolloverFees()` and `migrateRolloverFeesV2()`, the new rollover fees are compared against the global constant `MAX_ROLLOVER_FEE_PER_BLOCK` instead of the pair-specific `maxRolloverFeePerBlock`:

```
if (
    value.maxRolloverFeePerBlock > MAX_ROLLOVER_FEE_PER_BLOCK
        || value.lastLongPure.abs() > MAX_ROLLOVER_FEE_PER_BLOCK
        || value.brokerPremium > MAX_BROKER_PREMIUM_PER_BLOCK
```

```
) {
    revert WrongParams();
}
```

```
if (brokerPremiums[i] > MAX_BROKER_PREMIUM_PER_BLOCK || lastLongPures[i].abs() >
MAX_ROLLOVER_FEE_PER_BLOCK) {
    revert WrongParams();
}
```

Recommendation: Check the new rollover fees against `maxRolloverFeePerBlock` instead of `MAX_ROLLOVER_FEE_PER_BLOCK`, and revert if they exceed the maximum.

[L-03] Exposure limits check uses incorrect collateral amount

The exposure limits check in `getOpenTradeMarketCancelReason()` and `getAutomationOpenOrderCancelReason()` functions uses the full collateral amount without deducting oracle and builder fees, potentially causing incorrect trade cancellations. The `withinExposureLimits()` function is called with the original collateral amount, but oracle and builder fees are subtracted from collateral later in the trade execution process, making the check inaccurate.

This could result in trades being incorrectly cancelled for exceeding exposure limits when the actual effective collateral (after fee deductions) would be within acceptable limits, leading to unnecessary trade rejections.

Consider calculating and subtracting oracle and builder fees from the collateral amount before passing it to the `withinExposureLimits()` function.

[L-04] `getPairRolloverFees()` returns outdated accumulated fees

The `getPairRolloverFees()` function returns outdated `accPerOiLong` and `accPerOiShort` values unless `storeAccRolloverFees()` was called in the same block before the call. This means the function provides stale rollover fee data that doesn't include pending accumulated rollover fees for the current block.

While `getPendingAccRolloverFees()` exists as a public function to get pending fees, it only returns the accumulated value for either long or short side and doesn't return the other variables in the `pairRolloverFeesV2` mapping.

Consider updating `getPairRolloverFees()` to include pending fees in its return values.

[L-05] Underflow risk in `unregisterTrade()` when rollover fees equals collateral

When closing a trade, the function `unregisterTrade()` in `OstiumTradingCallbacks.sol` tries to calculate how much USDC the trader has left after deducting fees:

```
uint256 usdcLeftInStorage = collateralToClose - liquidationFee - rolloverFees;
```

But if the rollover fee hits its maximum cap (equal to the trade's collateral *), this calculation fails and causes the function to revert. Because of that, `unregisterTrade()` and everything that depends on it (like `closeTradeMarketCallback`) will be blocked.

Note maximum cap for rollover fee: 1. `closeTradeMarketCallback` calls. 2. `getTradeandPriceData` (to fetch rollover fee) calls. 3. `getTradeValue` calls. 4. `getTradeRolloverFeePure`. Inside `getTradeRolloverFeePure`, the rollover fee is calculated like this:

```
    function getTradeRolloverFeePure(
        uint256 accRolloverFeesPerCollateral,
        uint256 endAccRolloverFeesPerCollateral,
        uint256 collateral,
        uint32 leverage
    ) public pure returns (uint256) {
        if (endAccRolloverFeesPerCollateral <= accRolloverFeesPerCollateral) {
            return 1;
        }
        uint256 rolloverFee = ((endAccRolloverFeesPerCollateral - accRolloverFeesPerCollateral)
* collateral * leverage)
            / PRECISION_18 / PRECISION_2;

@>      return (rolloverFee > collateral) ? collateral : rolloverFee;
    }
```

This means the rollover fee can grow and is capped at `collateral`, and as a result, trade close transactions would fail if the rollover fee grows and becomes as much as collateral, and those trades would be stuck in the system as open trades.

Recommendations

Prevent revert when `fees ≥ collateral` to ensure the rest of the flow still unregisters the trade.

[L-06] Incorrect assignment of `lastUpdateBlock` in `migrateRolloverFeesV2()`

The variable `lastUpdateBlock` is a critical field used in rollover fee calculations.
It is referenced in for calculating accumulated rollover fees since the last update:

```
    function getPendingAccRolloverFees(uint16 pairIndex, bool long) public view returns
(uint256) {
        PairRolloverFeesV2 memory r = pairRolloverFeesV2[pairIndex];

        int256 currentAccRolloverFee = long ? r.accPerOiLong : r.accPerOiShort;
@>      uint32 blockDelta = ChainUtils.getBlockNumber().toUint32() - r.lastUpdateBlock;
        int256 rolloverFeePerBlock = (long ? r.lastLongPure : -r.lastLongPure) +
int256(r.brokerPremium);
        currentAccRolloverFee +=
```

```
            int256(rolloverFeePerBlock > 0 ? rolloverFeePerBlock : int256(0)) *
int256(uint256(blockDelta));

        return uint256(currentAccRolloverFee);
    }
```

The function `migrateRolloverFeesV2()` inside `OstiumPairInfo.sol`, is designed to migrate rollover fee state into the upgraded contract while preserving consistency of past state. However, in this function, the value of `lastUpdateBlock` is incorrectly set to the current block number.

```
pairRolloverFeesV2[pairId].lastUpdateBlock = ChainUtils.getBlockNumber().toUint32();
```

This breaks the intended rollover fee migration logic since `lastUpdateBlock` should instead be migrated from the old rollover struct (`oldR.lastUpdateBlock`).

It will cause under-charging of traders after migration, the pending rollover fee will be lost, and this will effect all the pairs.

Recommendations

Update `migrateRolloverFeesV2()` to assign `lastUpdateBlock` from the old rollover struct (`oldR.lastUpdateBlock`) or calculate the pending rollover and update the `accPerOi` before migrating.

[L-07] `pairMinLevPos` can be bypassed

When a user wants to open a trade, they call `openTrade()` in **OstiumTrading**, which then uses `getOpenTradeRevert()` to verify that the trade configuration is set correctly.

```
function getOpenTradeRevert(
        IOstiumTradingStorage storageT,
        IOstiumPairsStorage pairsStored,
        address sender,
        IOstiumTradingStorage.Trade memory t,
        uint256 maxAllowedCollateral
    ) external view {

        ///code...

        uint256 oracleFee = pairsStored.pairOracleFee(t.pairIndex);

        if (oracleFee > t.collateral) {
            revert IOstiumTrading.BelowOracleFee();
        }

@>      if ((t.collateral - oracleFee) * t.leverage / 100 <
pairsStored.pairMinLevPos(t.pairIndex)) {
            revert IOstiumTrading.BelowMinLevPos();
        }

       ///code...
    }
```

The system checks if `collateral - fees` is greater than `pairMinLevPos`. The problem is that the `builderFee` is not considered, which can allow a position to be opened with a value smaller than `pairMinLevPos`.

To illustrate the issue more clearly, let's consider an example.

- `pairMinLevPos = 900e18`
- `oracleFee = 10e18`
- Bob opens a position with `100e18` collateral, leverage = 1000 (x10), and `builderFee = 10%` (is for example; max is 0.5%).
- `pairMinLevPos` is checked using `(collateral - oracleFee) * leverage / 100 = (100e18 - 10e18) * 1000 / 100 = 900e18`, so the trade is allowed.
- In reality, the trade's effective value is `(100e18 - 10e18 (oracleFee) - 10e18 (builderFee)) * 10 = 890e18`, which is below `pairMinLevPos`. This would cause the transaction to revert with the `BelowMinLevPos()` custom error.

Recommendations

To solve the problem, subtract the `builderFee` when checking against `pairMinLevPos`, and revert the transaction if the resulting value is below the minimum.

[L-08] Trades fail as `registerTrade()` underflows from `builderFee`

When a user wants to open a trade, they call `openTrade()` in **OstiumTrading**, which then uses `getOpenTradeRevert()` to verify that the trade configuration is set correctly.

```
function getOpenTradeRevert(
        IOstiumTradingStorage storageT,
        IOstiumPairsStorage pairsStored,
        address sender,
        IOstiumTradingStorage.Trade memory t,
        uint256 maxAllowedCollateral
    ) external view {

       ///code...

        uint256 oracleFee = pairsStored.pairOracleFee(t.pairIndex);

        if (oracleFee > t.collateral) {
            revert IOstiumTrading.BelowOracleFee();
        }

        if ((t.collateral - oracleFee) * t.leverage / 100 <
pairsStored.pairMinLevPos(t.pairIndex)) {
            revert IOstiumTrading.BelowMinLevPos();
        }
```

It checks that the collateral supplied by the user is greater than the `oracleFee`, but this alone does not guarantee that the trade can be completed correctly, since another fee, the `builderFee`, may also be applied.

When trades are executed, they are handled by a keeper, which calls either `openTradeMarketCallback()` or `executeAutomationOpenOrderCallback()`, both of which execute `registerTrade()`.

```
function registerTrade(uint256 tradeId, IOstiumTradingStorage.Trade memory trade, uint256
latestPrice)
        private
        returns (IOstiumTradingStorage.Trade memory)
    {

       ///code...

        uint256 oracleFee = pairsStorage.pairOracleFee(trade.pairIndex);

        storageT.handleOracleFee(oracleFee);

        trade.collateral -= oracleFee;
        emit OracleFeeCharged(tradeId, trade.trader, oracleFee);

        if (trade.builder != address(0) && trade.builderFee > 0) {
            uint256 builderFee = trade.builderFee * tradeNotional / PRECISION_6 / 100;
            storageT.transferUsdc(address(storageT), trade.builder, builderFee);
@>          trade.collateral -= builderFee;
            emit BuilderFeeCharged(tradeId, trade.trader, trade.builder, builderFee);
        }

      ///code...
    }
```

As you can see, it first subtracts the `oracleFee`, and if a builder is set, it then subtracts the `builderFee`. If `oracleFee + builderFee > collateral`, the transaction will always revert due to underflow, and the trade can never be executed.

Recommendations

To resolve this issue, ensure that `collateral >= oracleFee + builderFee` in `getOpenTradeRevert()`, and revert if the condition is not met.